A Midterm Progress Report

on

Semantic Retrieval and Contextual Exploration System

Submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY
(Computer Science and Engineering)

SUBMITTED BY:

Balkaran Singh (2302494)
Gurpreet Kaur (2302530)
Gurveer Singh (2302533)

UNDER THE GUIDANCE OF

Dr. Parminder Singh

(October 2026)



Department of Computer Science and Engineering
GURU NANAK DEV ENGINEERING COLLEGE
LUDHIANA, 141006

Table of Contents

Contents	Page No.
1. Introduction	1
2. System Requirements	4
3. Software Requirement Analysis	6
4. Software Design	9
5. Testing Module	19
6. Performance of the Project Developed	22
7. Output Screens	31
8. References	34

List of Figures

List of Figures

1 System architecture of the Gurbani semantic retrieval system 9
2 Data preprocessing pipeline 10
3 Data flow diagram, level 0 11
4 Data flow diagram, level 1 12
5 Use case diagram 13
6 Sequence diagram for a semantic retrieval request 14
7 Activity diagram for the model training process 15
8 Module and component diagram 16
9 Deployment diagram 16
10 Data model of the processed corpus 17
11 Contrastive learning objective with a mined hard negative 18
12 Structure of the raw dataset with Pankatis and their commentaries 25
13 Cleaned dataset showing the derived flag columns; the final two rows are structural heading lines 25
14 Mined training triplets of anchor, positive and hard negative 25
15 Dataset profile report produced by the preprocessing module 26
16 Training pair construction report showing the leakage-free partition 27
17 Report of the mined evaluation benchmark 27
18 Report of the mined Pankati-to-Pankati training pairs 28
19 Report of the mined hard negatives 28
20 Implementation of the structural line detector 29
21 Implementation of the leakage-free partitioning procedure 29
22 Implementation of the Pankati-to-Pankati pair mining 30
23 Implementation of the LoRA fine-tuning procedure 30
24 Retrieval output for a query Pankati, showing similarity score, Ang, Raag, Shabad and Steek for each result 31
25 Side-by-side output of the pretrained and fine-tuned models for the same query . 32
26 Fine-tuning run showing the falling training and validation loss 32
27 Evaluation output comparing all retrieval systems on the benchmark 33
28 Corpus encoding with shard checkpointing and progress reporting 33

1. Introduction

Semantic understanding is an important area of Natural Language Processing (NLP) that focuses on identifying the meaning and intent conveyed by a piece of text rather than simply matching individual words. Traditional text retrieval techniques often depend on keywords, exact matching, or surface-level lexical similarity, which can fail when two texts express the same idea using different vocabulary or sentence structures. Context understanding addresses this limitation by considering the relationship between words and the overall meaning of a sentence or passage. Recent advances in deep learning and language representation models have enabled text to be represented as contextual embeddings, where semantically similar texts can be positioned close to each other in a shared vector space [1, 2].

This problem of contextual and semantic understanding becomes particularly significant when working with Gurbani. Sri Guru Granth Sahib contains a vast collection of Gurbani written in Gurmukhi, where the meaning of a Pankati (line) is often dependent not only on individual words but also on the overall spiritual and contextual message conveyed by it. A literal word-level interpretation may therefore be insufficient for identifying other Pankatis that express a similar teaching or spiritual idea. The project addresses this limitation by developing an AI-based system capable of understanding the contextual meaning, or *Bhavarth*, of a Gurbani Pankati and retrieving other Pankatis that convey a similar spiritual or contextual idea.

The central difficulty is that the same teaching may be expressed in entirely different vocabulary. For example, the Pankati *"ਵਿਸਰੀ ਤਿਸੈ ਪਰਾਈ ਤਾਤਾ"* (Ang 189) and the Pankati *"ਛਾਡਿ ਵਿਡਾਣੀ ਤਾਤਿ ਮੁੜੇ"* (Ang 889) both teach the abandonment of envy towards others, yet they share no common word and are separated by seven hundred Angs. No keyword-based search system can connect them. A system that understands meaning, however, should place them close together.

1.1 Work Completed in This Phase

During the current phase of the project, the following work has been completed. A structured Gurbani dataset of 60,360 Pankatis with five independent commentaries (Steeks) was profiled and cleaned. A leakage-free training and evaluation split was constructed. An automatic evaluation benchmark of 827 verse-to-verse queries was mined from the commentaries. A baseline was established using the pretrained BGE-M3 embedding model and two keyword-search baselines. The embedding model was then fine-tuned on Gurbani-specific training pairs using Low-Rank Adaptation (LoRA) [3], and the improvement was measured against the same benchmark.

1.2 Objectives

Following are the objectives for the project work:

1. To develop a semantic representation framework for capturing the contextual meaning (*Bhavarth*) of Gurbani text.
2. To develop a semantic retrieval system for identifying and ranking Gurbani Pankatis based on contextual and thematic similarity.
3. To develop an intelligent Gurbani information system integrating semantic retrieval with Gurbani metadata, Raag information, and relevant digital resources.

1.3 Elaboration of the Objectives

Objective 1: Semantic representation framework. This objective requires that a Gurbani Pankati be converted into a numerical representation that encodes its meaning rather than its spelling. A pretrained multilingual embedding model [4] cannot do this on its own, because it has never been exposed to Gurbani. The objective is therefore met by adapting such a model using supervision derived from the Steeks written by traditional commentators. Each Pankati is aligned with the explanations given for it, so that verses explained in similar terms acquire similar representations.

Objective 2: Semantic retrieval and ranking. Once every Pankati is represented as a vector, retrieval becomes a nearest-neighbour search in the vector space [5, 6]. This objective requires that the ranking produced by the system be measurably better than keyword search. To establish this, an evaluation benchmark and two keyword baselines were constructed, and all systems were scored on identical queries.

Objective 3: Intelligent Gurbani information system. Retrieved Pankatis are of limited use without context. This objective requires that each result be presented with its Ang number, Raag, Shabad identifier, and the Steeks that explain it, and that results be drawn from different Shabads so that the user discovers Gurbani beyond the passage already in front of them.

2. System Requirements

The project has two distinct computational phases with different requirements. Data preparation, retrieval, and evaluation are light enough for a standard laptop, whereas fine-tuning a 568-million-parameter transformer model requires a CUDA-capable graphics processor. Both configurations are described below, since both were used during development.

2.1 Hardware Requirements

Table 1: Hardware requirements

Component	**Development Machine**	**Training Machine**
Processor	Intel Core i3-1005G1, 2 cores / 4 threads, 1.20 GHz	Multi-core desktop processor
Memory (RAM)	11.8 GB	16 GB
Graphics	Intel UHD Graphics (integrated, no CUDA)	NVIDIA GeForce RTX 3060, 12 GB VRAM
Storage	5 GB free space for datasets, embeddings and model files	10 GB free space
Connectivity	Stable internet for downloading pretrained models	Stable internet

The distinction between the two machines is not incidental. Fine-tuning BGE-M3 with the Adam optimiser requires approximately sixteen bytes per parameter for the weights, their gradients and the optimiser state, which amounts to roughly nine gigabytes before any activations are stored. This does not fit comfortably within the memory available on the development laptop, and in the absence of a CUDA device every operation would execute on two low-clock CPU cores. The measured cost of a single forward pass over the corpus on the development machine was 112 minutes; a training run, which requires both forward and backward passes over a far larger number of samples, would therefore be impractical. Training was consequently performed on the RTX 3060, where the same work completes in minutes.

2.2 Software Requirements

Table 2: Software requirements

Software	Version	Purpose
Windows	10 / 11	Operating system
Python	3.11	Implementation language
PyTorch	2.14 (CPU and CUDA builds)	Deep learning framework
Sentence-Transformers	6.0.1	Embedding model training and inference
Transformers	5.16.1	Underlying model architecture
PEFT	0.21.0	Low-Rank Adaptation (LoRA)
FAISS	1.15.0	Vector similarity search
scikit-learn	1.9.0	TF–IDF based pair mining
pandas / NumPy	3.0.5 / 2.4.6	Data processing
rank_bm25	0.2.2	Keyword-search baseline
openpyxl	3.1.5	Reading the source workbook
python-dotenv	1.2.3	Configuration management
Visual Studio Code	Latest	Development environment
Git	Latest	Version control

2.3 Pretrained Model Requirement

The system uses **BGE-M3** (`BAAI/bge-m3`) [4] as the base embedding model. It is a multilingual text-embedding model built on the XLM-RoBERTa-large architecture, containing 568 million parameters arranged in 24 transformer layers, and producing embeddings of 1024 dimensions. The model is approximately 2.3 GB in size and is downloaded once from the Hugging Face model repository. It was selected because it supports more than one hundred languages including Punjabi, handles long input sequences, and is designed specifically for retrieval rather than classification.

2.4 Dependency Management

Because the project is developed across machines with and without a CUDA device, the dependency specification is split into three files. A shared file lists the version-independent packages; a CPU file adds the CPU build of PyTorch; and a GPU file adds the CUDA build together with the additional packages required for training. A single requirements file cannot serve both configurations, since installing the CUDA build on a machine without an NVIDIA device wastes approximately 2.5 GB, while installing the CPU build on the training machine would make the graphics processor unusable.

3. Software Requirement Analysis

3.1 Problem Definition

Given a Gurbani Pankati as input, the system must return other Pankatis from Sri Guru Granth Sahib that convey the same contextual meaning (*Bhavarth*), together with their metadata and commentaries. The returned Pankatis must be ranked by semantic relevance, and the ranking must remain correct when the related Pankatis share no vocabulary with the input.

The problem has three properties that make it difficult:

(a) **Absence of ground truth.** There exists no labelled dataset stating that one Pankati is semantically similar to another. Supervision must therefore be derived from an indirect source.

(b) **Vocabulary mismatch.** Measurements on the prepared corpus show that among Pankatis judged semantically related, a substantial proportion share no words at all. Lexical retrieval cannot reach these cases even in principle.

(c) **Absence of a Gurbani-aware model.** General-purpose multilingual models [7–9] are trained on contemporary web text. They have no exposure to the archaic Gurmukhi of Sri Guru Granth Sahib and consequently fall back on surface-level similarity.

3.2 Key Insight Underlying the Solution

The problem of missing ground truth is solved by observing that the semantic work has already been performed by traditional scholars. The dataset contains, for each Pankati, explanations written in modern Punjabi prose by several independent commentators. Two Pankatis that mean the same thing are explained in very similar words, even when the Pankatis themselves share no vocabulary. The following example illustrates this:

Table 3: The semantic problem is difficult on the verses and easy on the commentaries

Pankati A	ਵਿਸਰੀ ਤਿਸੈ ਪਰਾਈ ਤਾਤਾ
Pankati B	ਛਾਡਿ ਵਿਡਾਣੀ ਤਾਤਿ ਮੂੜੇ
Shared words	none
Steek of A	ਉਸ ਨੂੰ ਹੋਰਨਾਂ ਨਾਲ ਈਰਖਾ ਕਰਨੀ ਭੁੱਲ ਜਾਂਦੀ ਹੈ
Steek of B	ਹੇ ਮੂਰਖ! ਦੂਜਿਆਂ ਨਾਲ ਈਰਖਾ ਕਰਨੀ ਛੱਡ ਦੇ
Shared words	ਈਰਖਾ, ਕਰਨੀ, ਨਾਲ

The semantic relationship that is invisible at the level of the verses becomes a simple word-overlap count at the level of the commentaries. This observation is used twice in the system: to generate training supervision, and to generate an evaluation benchmark.

3.3 Functional Requirements

Table 4: Functional requirements

ID	Requirement
FR-1	The system shall accept a Gurbani Pankati as input, with or without the terminating danda and verse numbers.
FR-2	The system shall accept multiple Pankatis, or a complete Shabad, as a single query.
FR-3	The system shall return the top k semantically related Pankatis, ranked by similarity.
FR-4	The system shall exclude Pankatis belonging to the same Shabad as the query.
FR-5	The system shall return at most one Pankati per Shabad, so that results are diverse.
FR-6	The system shall display the Ang number, Raag, Shabad identifier and Steek for every retrieved Pankati.
FR-7	The system shall support comparison between two models in a single run.
FR-8	The system shall exclude structural heading lines from the searchable corpus.
FR-9	The system shall treat a Pankati that repeats in the scripture as one searchable entry.

3.4 Non-Functional Requirements

Table 5: Non-functional requirements

ID	Requirement
NFR-1	A query against a Pankati already present in the corpus shall be answered without loading the embedding model.
NFR-2	Corpus embedding shall be resumable, so that an interrupted run does not restart.
NFR-3	All file paths and hyper-parameters shall be configurable without modifying source code.
NFR-4	Training runs shall be reproducible through fixed random seeds.
NFR-5	The source dataset shall never be modified; every stage shall write new files.
NFR-6	Evaluation shall be free of data leakage between training and test partitions.

3.5 Modules and Their Functionalities

The system is decomposed into six modules. Each module is implemented as an independent script that reads the output of the previous stage and writes new files, so that any stage can be re-executed without repeating the others.

Module 1 — Data Preprocessing (`clean_dataset.py`). Reads the source workbook, normalises whitespace and removes markup, identifies structural heading lines, detects Pankatis that repeat within the scripture, counts the commentaries available for each Pankati, and writes a cleaned dataset together with a profile report and an audit file of every excluded row. The original workbook is opened read-only.

Module 2 — Training Pair Construction (`create_training_pairs.py`). Partitions the cleaned dataset into training, validation and test sets using a grouping procedure that prevents leakage, generates Pankati–Steek training pairs, and writes the deduplicated retrieval corpus with its metadata and commentaries attached.

Module 3 — Benchmark Construction (`build_eval_sets.py`). Mines an evaluation benchmark of verse-to-verse queries from the test partition by identifying Pankati pairs that two independent commentators explain in similar terms.

Module 4 — Training Data Mining (`mine_verse_pairs.py`, `mine_hard_negatives.py`). Mines Pankati-to-Pankati positive pairs from the training partition using the same two-commentator rule, and then mines hard negatives, that is, Pankatis which share substantial vocabulary with the anchor but are explained differently.

Module 5 — Model Training (`train_bge.py`). Fine-tunes BGE-M3 on the mined pairs using Low-Rank Adaptation [3] and a contrastive objective, verifies that the trained model differs behaviourally from the base model, and saves the adapter.

Module 6 — Retrieval and Evaluation (`build_index.py`, `semantic_search.py`, `evaluate.py`, `diagnose_transfer.py`). Encodes the corpus into vectors, performs interactive retrieval with diversification and metadata display, and scores any number of retrieval systems on the benchmark.

4. Software Design

This chapter presents the detailed design of the system. It begins with the overall architecture, then describes the design of each stage of the pipeline, and finally presents the standard design diagrams required for the solution of the three objectives listed in Chapter 1.

4.1 System Architecture

The system is organised as an offline preparation pipeline and an online retrieval path. The offline pipeline cleans the dataset, constructs training data, trains the embedding model and encodes the corpus into a vector index. The online path accepts a query, embeds it, searches the index, applies diversification and presents the results with their metadata. The architecture is shown in Figure 1.

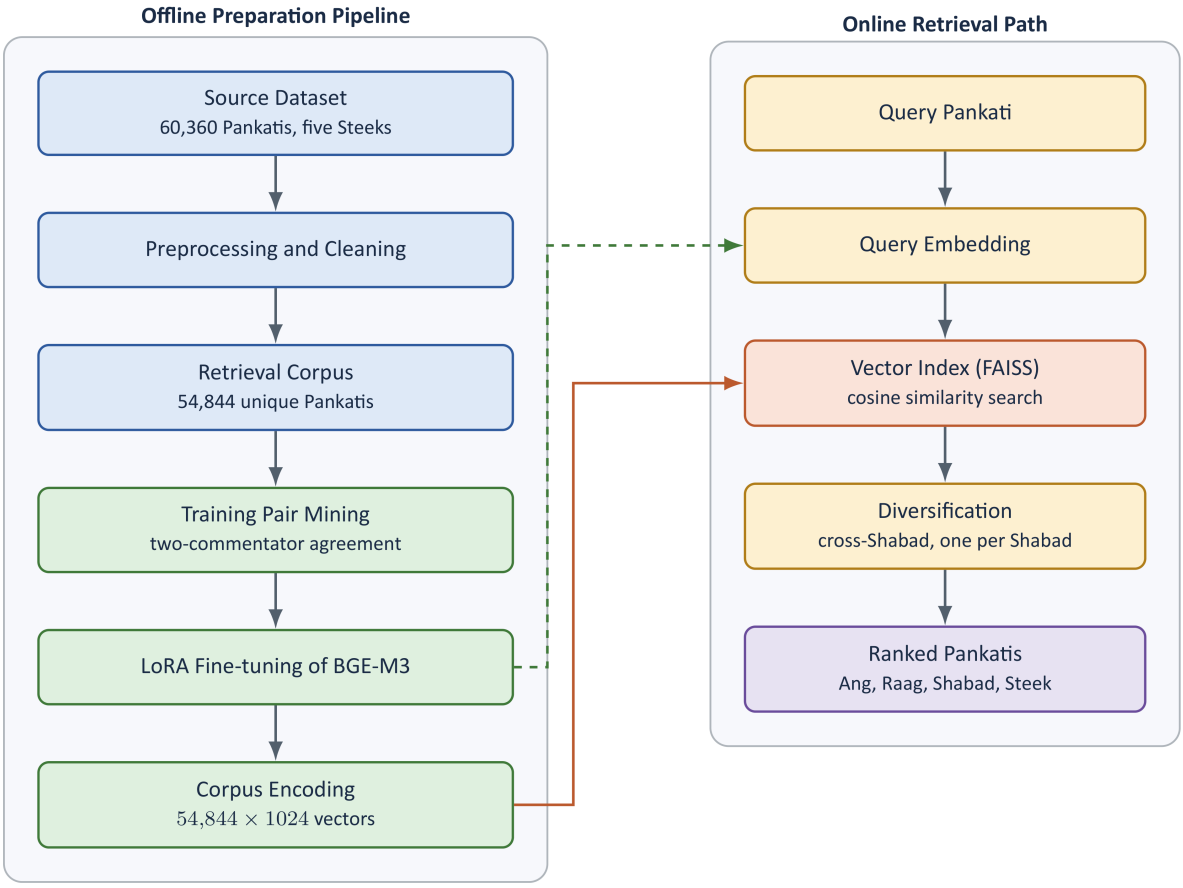


Figure 1: System architecture of the Gurbani semantic retrieval system

4.2 Design of the Data Preprocessing Stage

The preprocessing stage has to solve a problem that is not visible in the raw data. The source workbook contains a flag marking structural lines, but this flag identifies only 720 rows, whereas the corpus contains approximately five thousand such lines. Headings such as "ੴ ਸਤਿਗੁਰ ਪ੍ਰਸਾਦਿ" occur 522 times and "ਪਉੜੀ" occurs 515 times. These are section titles rather than teachings, and because they repeat verbatim they would also disturb contrastive training.

Two complementary detectors are therefore applied. The first matches prefix patterns

characteristic of headings. The second matches a vocabulary rule: a short line terminated by a danda whose every token is a raag name or a composition term is a heading, not a teaching. The raag vocabulary is derived from the project's own raag metadata file and compared on consonant skeletons so that spelling variants such as ਬਿਲਾਵਲੁ and ਬਿਲਾਵਲ are unified.

A scripture guard protects genuine verses from both detectors: a line of thirty characters or more that all three commentators chose to translate is a teaching, irrespective of the word it begins with. This guard rescues verses such as "ਰਾਗੁ ਨਾਦੁ ਸਭੁ ਸਚੁ ਹੈ ਕੀਮਤਿ ਕਹੀ ਨ ਜਾਇ", which would otherwise be removed by the raag-heading rule.

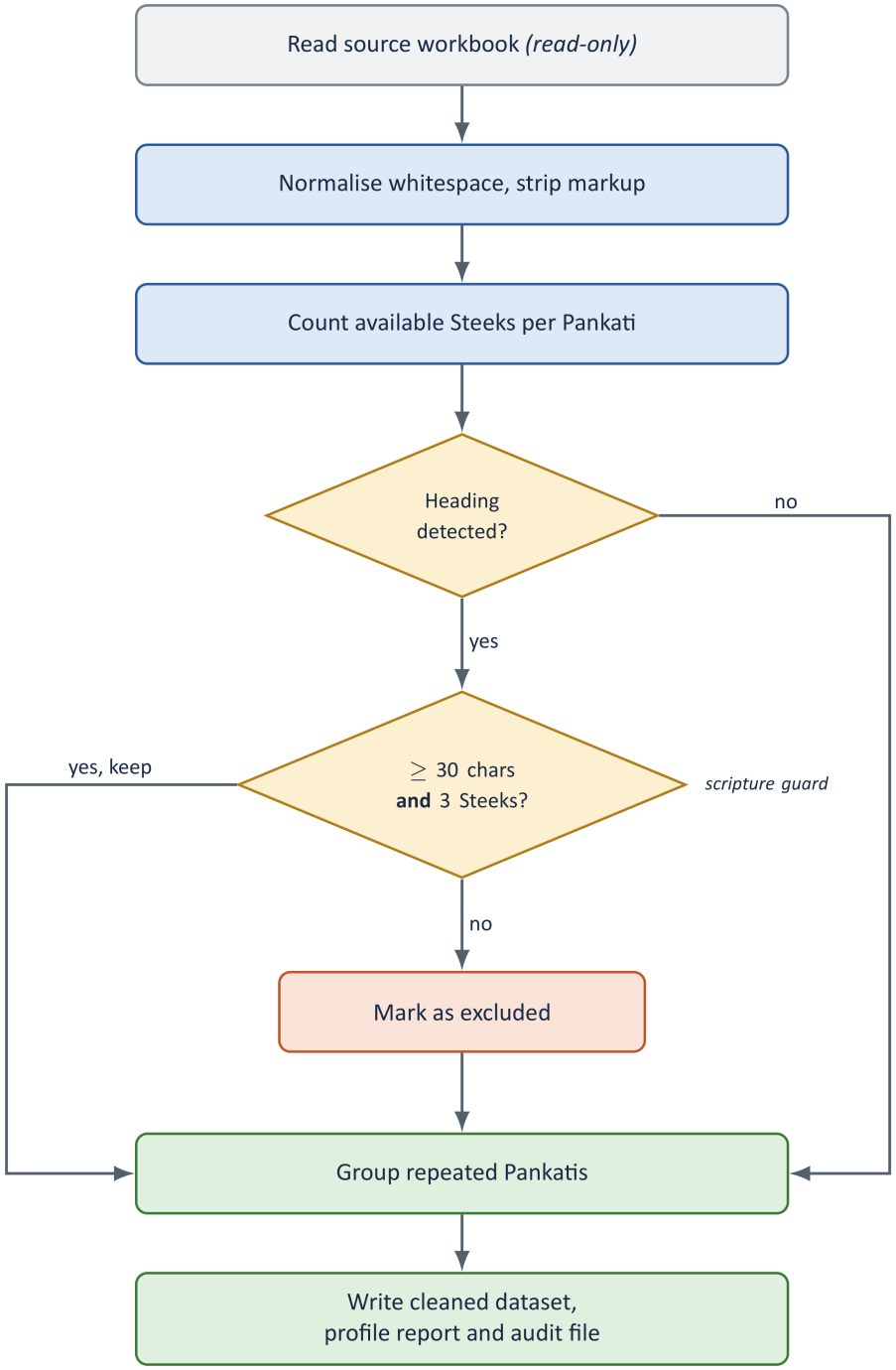


Figure 2: Data preprocessing pipeline

4.3 Data Flow Diagrams

4.3.1 Level 0 (Context Diagram)

At the highest level the system accepts a Pankati from the user and returns semantically related Pankatis together with their associated information. The scripture dataset and the pretrained model act as external data sources.

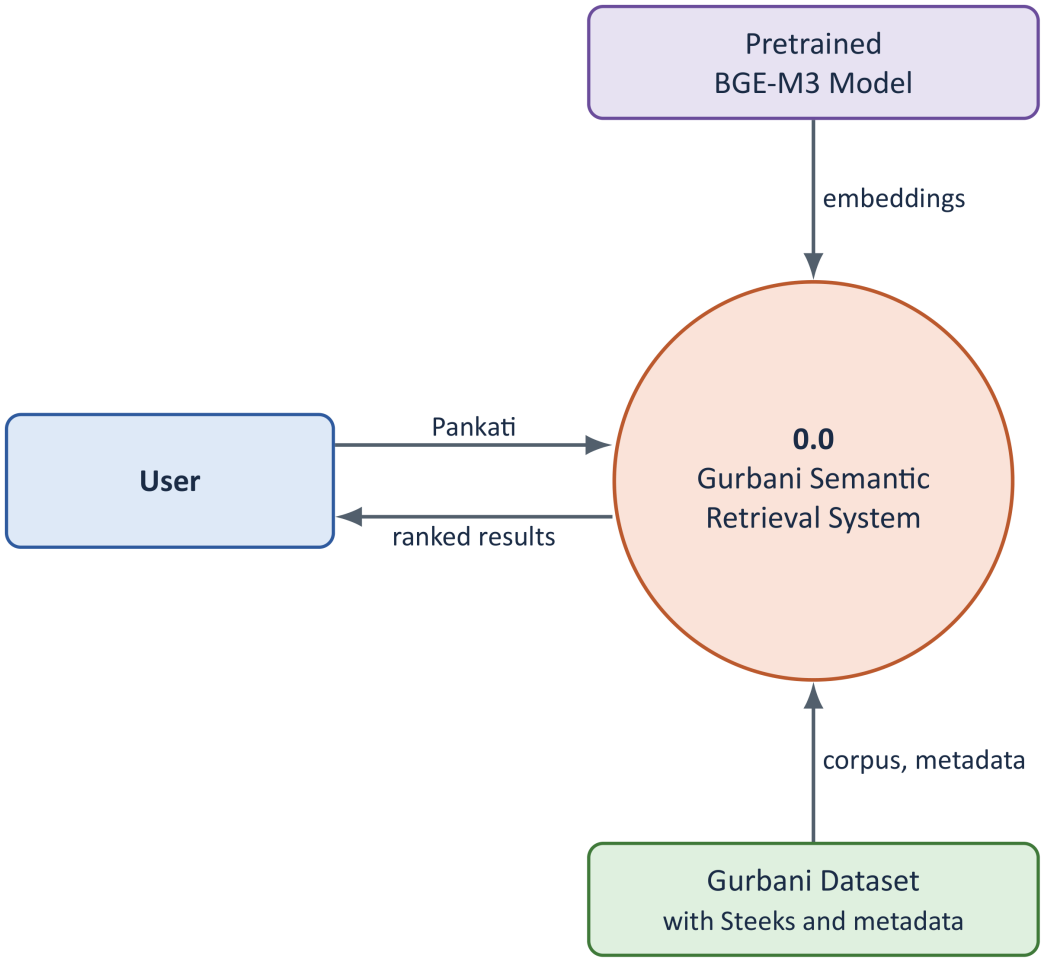


Figure 3: Data flow diagram, level 0

4.3.2 Level 1

The level 1 diagram expands the system into its processes and data stores.

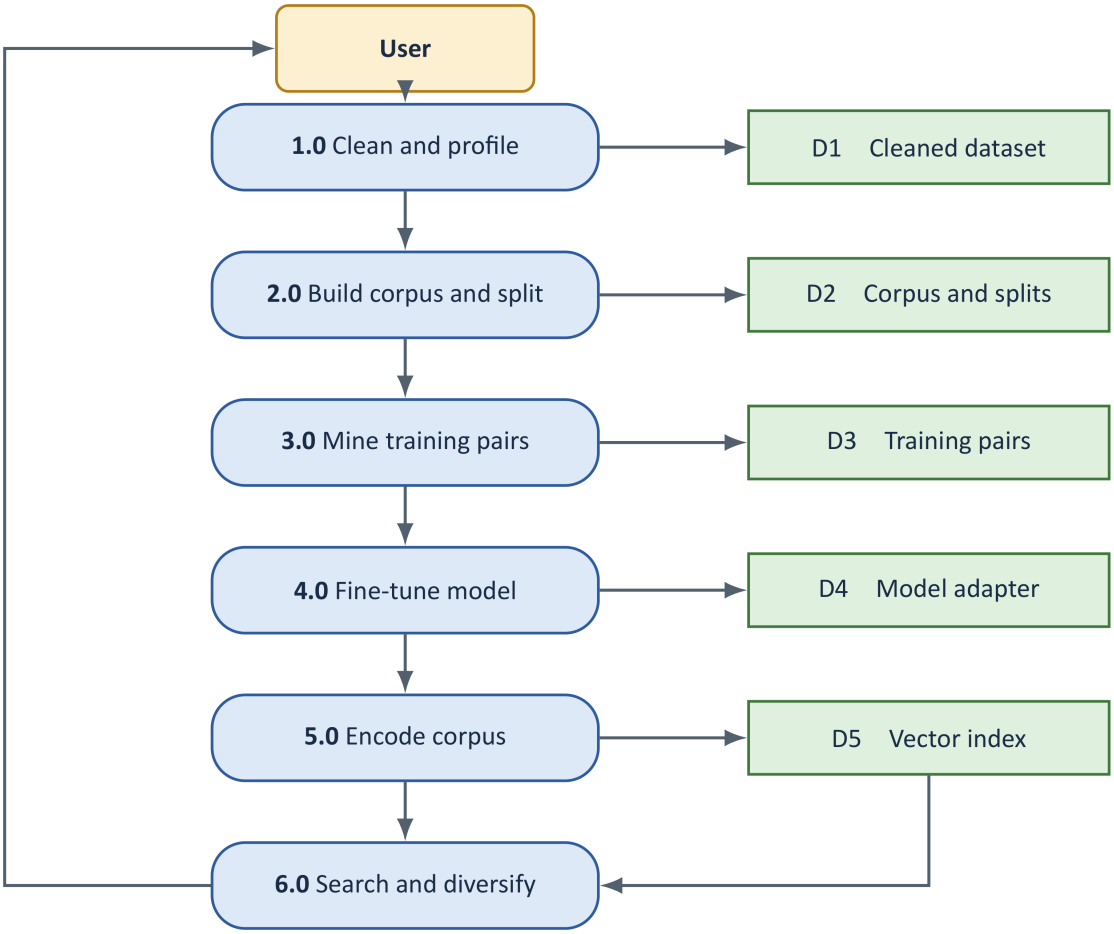


Figure 4: Data flow diagram, level 1

4.4 Use Case Diagram

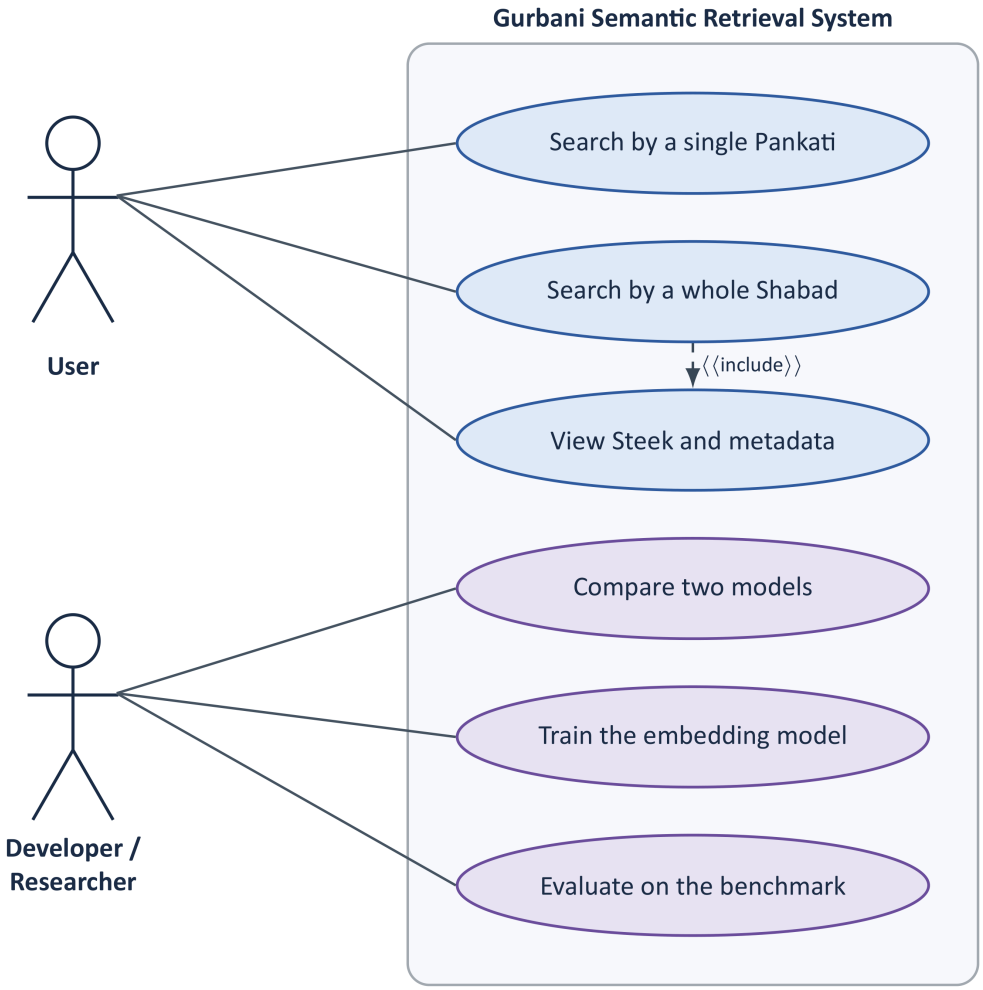


Figure 5: Use case diagram

4.5 Sequence Diagram

The sequence diagram in Figure 6 shows the interaction for a retrieval request. An important design decision is visible here: when the query is a Pankati that already exists in the corpus, its vector is read from the cached index rather than computed, so that the 2.3 GB model need not be loaded at all.

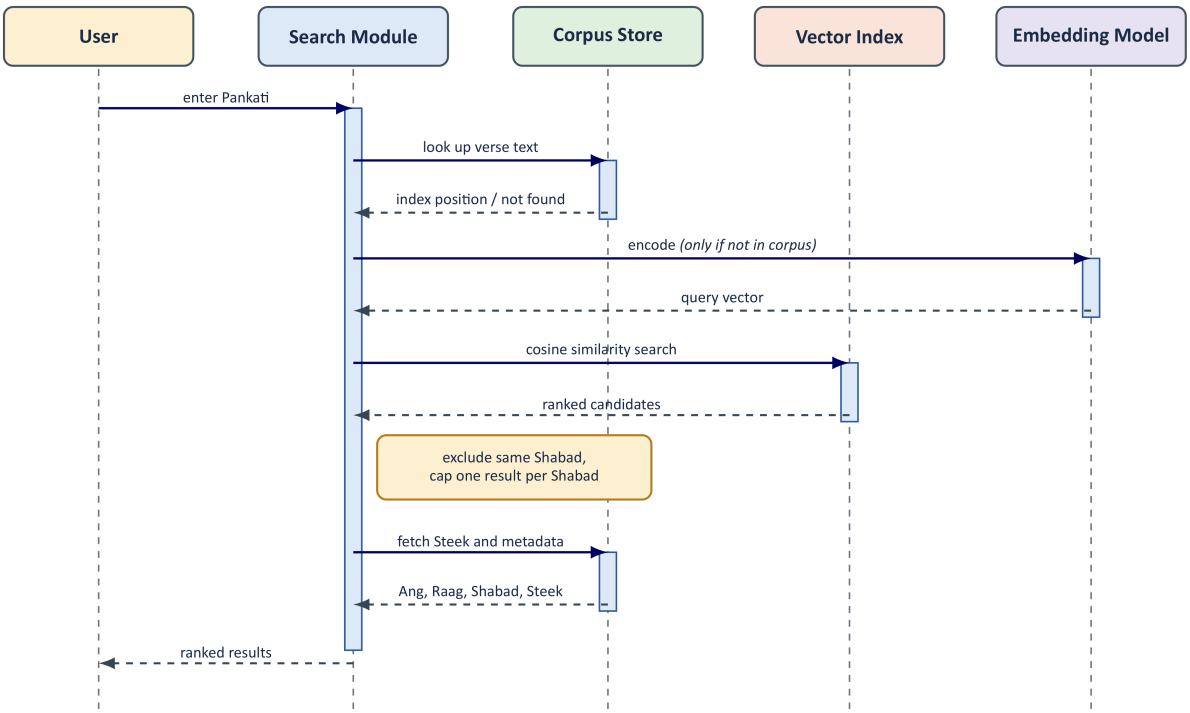


Figure 6: Sequence diagram for a semantic retrieval request

4.6 Activity Diagram for Model Training

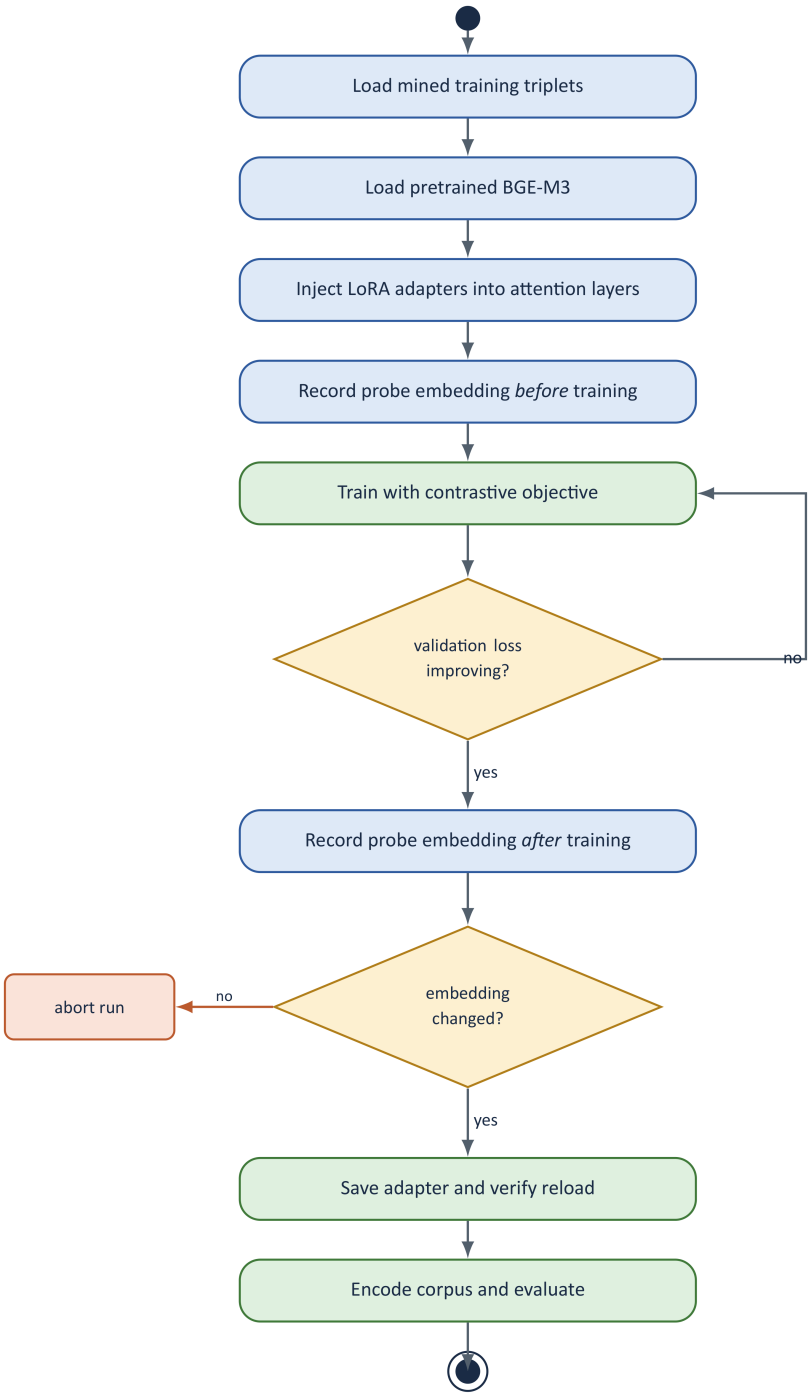


Figure 7: Activity diagram for the model training process

4.7 Module and Component Diagram

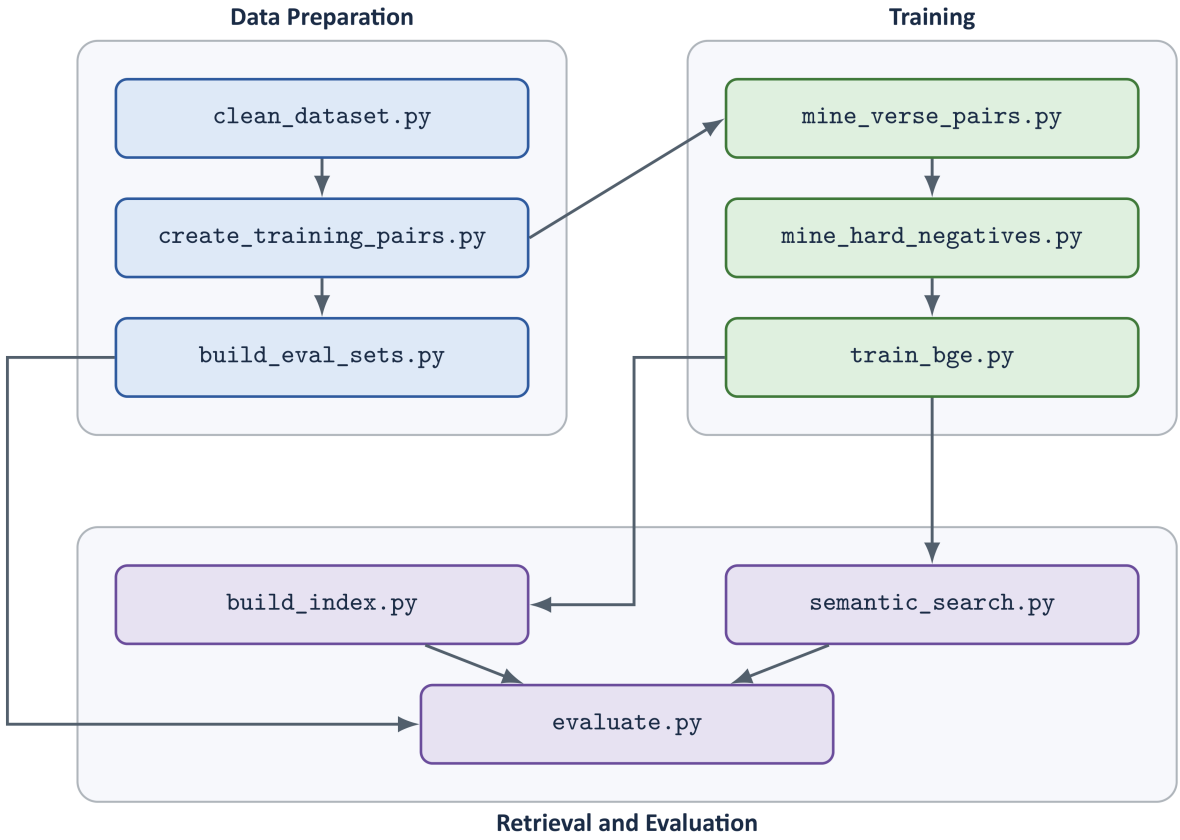


Figure 8: Module and component diagram

4.8 Deployment Diagram

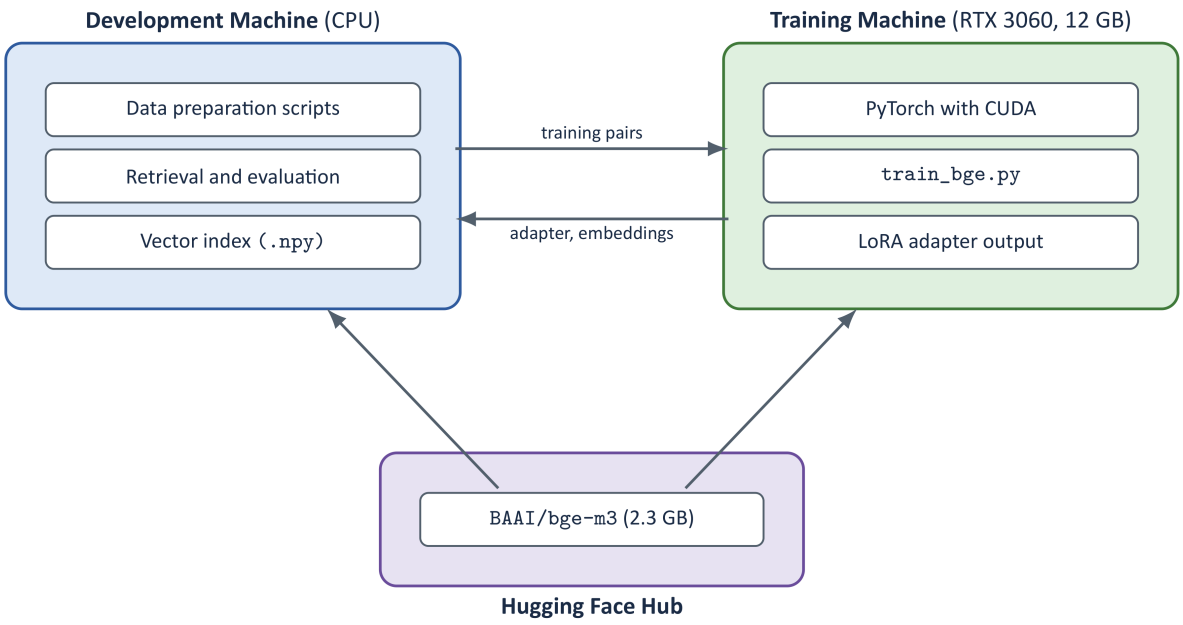


Figure 9: Deployment diagram

4.9 Data Design

The processed data is stored as files rather than in a relational database, because every stage is a batch transformation and no concurrent writes occur. The logical relationships between the entities are nevertheless those of a relational schema, and are shown in Figure 10.

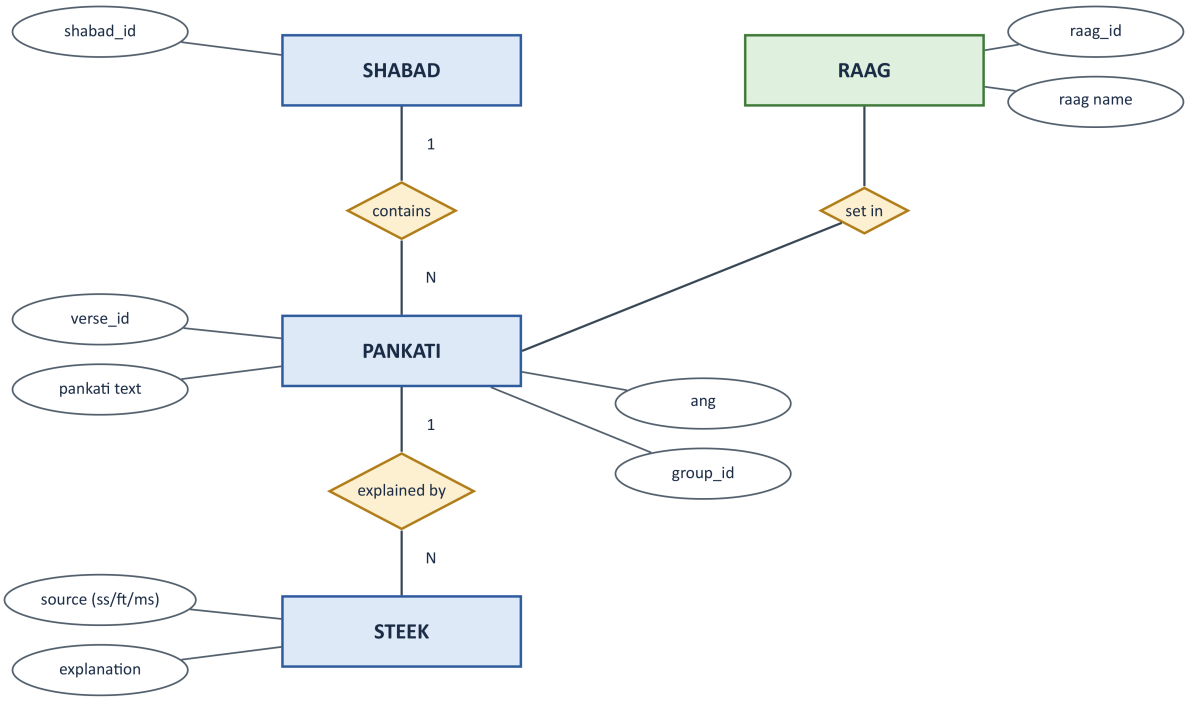


Figure 10: Data model of the processed corpus

Table 6: Principal data files produced by the pipeline

File	Format	Contents
cleaned_gurbani.csv	CSV	All rows with derived flag columns
corpus_pankatis.jsonl	JSON Lines	54,844 unique Pankatis with Steeks and metadata
training_pairs.jsonl	JSON Lines	Pankati–Steek training pairs
verse_pairs_hn.jsonl	JSON Lines	Pankati–Pankati triplets with hard negatives
eval_semantic_pairs.jsonl	JSON Lines	Evaluation benchmark
embeddings.npy	NumPy array	$54{,}844 \times 1024$ float32 vectors

4.10 Design of the Learning Objective

The model is trained with a contrastive objective [10]. For each training example the model is presented with an anchor Pankati, the Pankati that carries the same meaning, and a set of negatives. The objective increases the similarity between the anchor and its positive while decreasing the

similarity between the anchor and every negative. The negatives are of two kinds. In-batch negatives are the other Pankatis present in the same batch, which are inexpensive but easy to distinguish. Hard negatives are mined deliberately: Pankatis that share a large proportion of their vocabulary with the anchor but are explained differently by the commentators. These prevent the model from succeeding through keyword overlap alone.

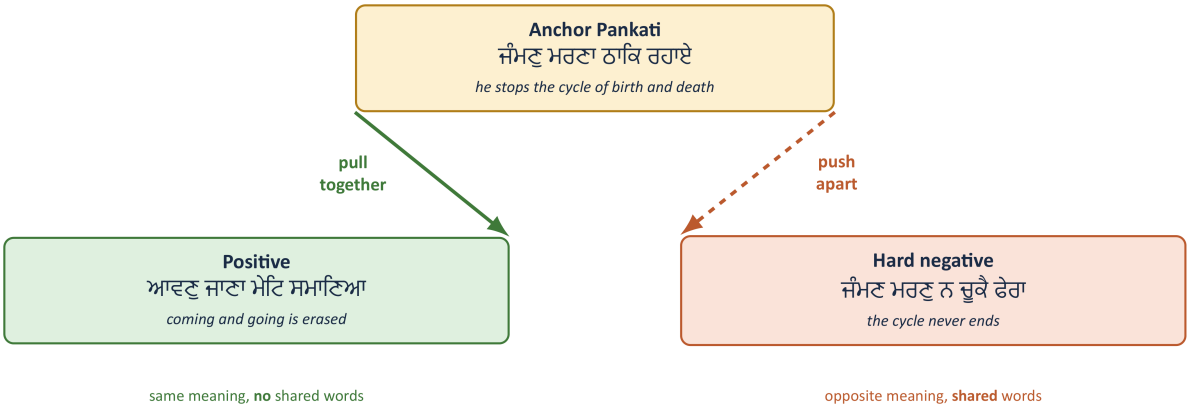


Figure 11: Contrastive learning objective with a mined hard negative

4.11 Design of the Evaluation Benchmark

Because no labelled verse-to-verse similarity data exists, the benchmark is mined. A pair of Pankatis is accepted as a gold pair when two independent commentators, Sahib Singh and Manmohan Singh, both explain them in similar terms. Requiring agreement between two authors of different periods and styles removes pairs that match only on formulaic phrasing such as the attribution "ਕਬੀਰ ਆਖਦਾ ਹੈ". Gold pairs are restricted to different Shabads, and word overlap between the two verses is recorded but never used as a filter, since the criterion is identity of meaning and not difference of wording.

4.12 Design Decisions Protecting the Validity of Results

Grouped partitioning. The dataset is not split at random. A union-find structure groups together every pair of rows that share a Shabad, that have identical Pankati text, or that have identical Steek text, and whole groups are assigned to a partition. Without this, consecutive lines of one Shabad would appear on both sides of the split and the measured improvement would reflect leakage rather than learning.

Exclusion of duplicated commentaries. Measurement showed that `steek_bdb` is byte-identical to `steek_ss` in 99.9 per cent of rows, and that `steek_pss` is a word-glossary rather than a paraphrase. Both are excluded from training so that no Pankati receives double weight and no glossary structure is learned, while both are retained in the corpus for display.

5. Testing Module

Testing a semantic retrieval system differs from testing a conventional application, because the correctness of a result is a matter of meaning rather than of a fixed expected value. The testing strategy therefore combines conventional software testing of the pipeline stages with quantitative retrieval evaluation and qualitative inspection.

5.1 Testing Techniques Proposed

Unit testing of text-processing rules. The heading detector and the tokeniser operate on Gurmukhi text, where incorrect behaviour is not visible by inspection. Each rule is therefore tested against a list of lines whose correct classification is known in advance, containing both headings and genuine verses chosen to be adversarial.

Integrity testing of the data pipeline. After each stage, the output is checked for structural correctness: row counts must reconcile, no partition may share a Shabad with another, and the number of embeddings must equal the number of corpus entries.

Leakage testing. Every pair of partitions is tested for overlap in Pankati text, Shabad identifier, leakage-group identifier and Steek text. All four counts must be zero.

Retrieval evaluation. The system is scored on the mined benchmark using standard information-retrieval measures: Hit@k, Mean Reciprocal Rank and normalised Discounted Cumulative Gain. Keyword baselines [11] are scored on identical queries so that differences are attributable to the retrieval method alone.

Ablation testing. Each design change is evaluated as a separate model and scored on the same benchmark, so that its contribution can be isolated.

Qualitative testing. A fixed list of query Pankatis with known correct answers is used for manual inspection of the ranked output.

5.2 Test Cases

Table 7: Test cases executed

ID	Test case	Expected result	Status
T-01	Classify ਪਉੜੀ ॥	Detected as heading	Pass
T-02	Classify ਛਕਾ ੧ ॥	Detected as heading	Pass
T-03	Classify ਆਸਾ ਮਹਲਾ ੫ ॥	Detected as heading	Pass
T-04	Classify the full Mool Mantar	Retained as scripture	Pass
T-05	Classify ਵਾਰ ਵਾਰ ਜਾਈ ਲਖ ਵਾਰੀਆ ॥	Retained as scripture	Pass
T-06	Classify ਰਾਗੁ ਨਾਦੁ ਸਭੁ ਸਚੁ ਹੈ ...	Retained by scripture guard	Pass
T-07	Cross-partition Pankati overlap	Zero	Pass
T-08	Cross-partition Shabad overlap	Zero	Pass
T-09	Cross-partition Steek overlap	Zero	Pass
T-10	Embedding count equals corpus size	54,844	Pass
T-11	Query without terminating danda	Matched to the corpus verse	Pass
T-12	Query with a partial Pankati	Matched to the corpus verse	Pass
T-13	Multi-verse query of one Shabad	Single averaged query vector	Pass
T-14	Results from the query's own Shabad	Excluded	Pass
T-15	Two results from one Shabad	At most one returned	Pass
T-16	Repeated Pankati in the corpus	One searchable entry	Pass
T-17	Saved model reload consistency	Cosine $= 1.000$	Pass
T-18	Training with unchanged weights	Run aborts with an error	Pass
T-19	Training script on a CPU machine	Clear message to use the GPU	Pass
T-20	Interrupted corpus encoding	Resumes from the last shard	Pass

5.3 Qualitative Test Queries

The following queries have known semantically related Pankatis that share no vocabulary with the query. They are used to inspect ranked output manually.

Table 8: Qualitative test queries with known related Pankatis

ID	Query Pankati	Expected related Pankati	Ang gap
Q-1	ਆਨ ਨ ਬੀਆ ਤੇਰੀ ਸਮਸਰਿ ॥	ਤੁਮ੍ਰੈ ਲਵੈ ਨ ਕੋਊ ਲਾਈ ॥	15
Q-2	ਵਿਸਰੀ ਤਿਸੈ ਪਰਾਈ ਤਾਤਾ ॥	ਛਾਡਿ ਵਿਡਾਣੀ ਤਾਤਿ ਮੂੜੇ ॥	700
Q-3	ਨਾਲਿ ਨਰਾਇਣੁ ਮੇਰੈ ॥	ਪ੍ਰਾਨੀ ਨਾਰਾਇਨ ਸੁਧਿ ਲੇਹਿ ॥	272
Q-4	ਤਿਸੁ ਗੁਰ ਕਉ ਸਦ ਜਾਉ ਬਲਿਹਾਰ ॥	ਹਰਿ ਵਿਟੜਿਅਹੁ ਸਦਾ ਘੋਲੇ ਜੀਉ ॥	979
Q-5	ਜੰਮਣੁ ਮਰਣਾ ਠਾਕਿ ਰਹਾਏ ॥	ਆਵਣੁ ਜਾਣਾ ਮੇਟਿ ਸਮਾਣਿਆ ॥	683
Q-6	ਸਚਾ ਵੇਪਰਵਾਹੁ ਇਕੋ ਤੂ ਧਣੀ ॥	ਸਾਚੇ ਠਾਕੁਰ ਵੇਪਰਵਾਹੇ ॥	1161

6. Performance of the Project Developed

6.1 Dataset Preparation Results

The source workbook contains 60,360 rows. Profiling established three facts that materially affected the design. First, `steek_bdb` is byte-identical to `steek_ss` in 54,941 of the 55,017 rows where both are present, a duplication rate of 99.9 per cent. Second, `steek_pss` contains an equals sign in 49,928 of 50,015 rows, confirming that it is a word-glossary rather than a paraphrase. Third, the supplied structural flag identifies only 720 rows whereas approximately five thousand heading lines are present.

Table 9: Dataset preparation results

Measure	**Value**
Total rows in source workbook	60,360
Rows excluded as structural or heading lines	5,077
Trainable rows retained	55,217
Unique Pankatis in the retrieval corpus	54,844
Pankati–Steek training pairs	131,806
Validation pairs	16,457
Test pairs	16,515
Leakage groups formed	5,190
Cross-partition overlap (all four checks)	0

6.2 Evaluation Benchmark

The benchmark consists of query Pankatis drawn from the held-out test partition, each with one or more gold Pankatis mined by two-commentator agreement. Gold pairs are cross-Shabad by construction. A substantial fraction of the gold pairs share no vocabulary with the query, and these form the subset on which keyword retrieval cannot succeed.

Table 10: Composition of the evaluation benchmark

Measure	**Value**
Benchmark queries	919
Gold pairs	1,874
Gold pairs with no shared vocabulary	679 (36.2%)
Mean Ang separation of a gold pair	395
Gold pairs in a different Raag	87.8%

6.3 Retrieval Results

Four systems were scored on identical queries against the same corpus of 54,844 Pankatis, with the query's own Shabad excluded from the results in every case. Hit@k is the percentage of

queries for which at least one gold Pankati appears within the top k results.

Table 11: Retrieval performance on the benchmark (all values are percentages)

System	H@1	H@5	H@10	H@50	MRR
BM25 (word level)	8.92	20.02	23.18	37.76	13.92
BM25 (character 3–5 gram)	9.47	20.46	27.09	44.07	15.01
BGE-M3, pretrained	7.83	15.13	19.37	30.25	11.51
BGE-M3, Steek-pair tuning	8.16	14.80	18.50	30.69	11.64
BGE-M3, verse-pair tuning	**11.43**	**21.65**	**28.29**	**45.27**	**16.77**

Three results deserve comment.

The pretrained model is worse than keyword search. BGE-M3 without adaptation achieves 19.37 per cent Hit@10, below the 27.09 per cent of character-level BM25. A state-of-the-art multilingual embedding model, applied directly, is outperformed by keyword matching on this task. This measurement justifies the need for Gurbani-specific adaptation, which the synopsis had assumed rather than demonstrated.

Training on the wrong objective produces no improvement. Fine-tuning on Pankati–Steek pairs changed Hit@10 from 19.37 to 18.50 per cent, which is within the margin of error for a benchmark of this size. A separate diagnostic established that the model had in fact learned: on the task it was trained for, retrieving a Pankati from its Steek, Hit@10 rose from 38.00 to 61.60 per cent. Aligning a verse with its own commentary therefore does not transfer to placing two semantically similar verses close together.

Training on verse pairs succeeds. Fine-tuning on mined Pankati-to-Pankati pairs raised Hit@10 from 19.37 to 28.29 per cent, an absolute improvement of 8.92 percentage points, and produced the best score of all five systems on every measure. This is the first configuration in which semantic retrieval outperforms keyword retrieval, which is the central claim of the project.

Table 12: Performance on gold pairs sharing no vocabulary with the query

System	Hit@10 on the zero-overlap subset
BM25 (word level)	0.00%
BM25 (character 3–5 gram)	1.19%
BGE-M3, pretrained	1.43%
BGE-M3, Steek-pair tuning	1.67%
BGE-M3, verse-pair tuning	**3.57%**

On the subset where the related Pankati shares no word with the query, verse-pair tuning gives a two and a half fold improvement over the pretrained model. The absolute values remain low, which indicates that substantial headroom remains; this subset is the hardest part of the

problem and is the focus of the remaining work.

6.4 Diagnostic Measurement

Table 13: Diagnostic separating learning from transfer

Task	Pretrained	After tuning
Steek → Pankati, Hit@10	38.00%	61.60%
Steek → Pankati, MRR	26.98	46.44
Pankati → Pankati, Hit@10	19.37%	18.50%

This diagnostic was decisive for the direction of the project. Without it, the absence of improvement on the target task would have been indistinguishable from a failure of the training procedure itself, and the natural response would have been to train longer or on more data. The measurement showed instead that training was highly effective but was optimising a different capability, and that the training objective rather than its scale had to change.

6.5 Computational Performance

Table 14: Computational performance

Operation	CPU machine	RTX 3060
Encoding 54,844 Pankatis	112 minutes	about 3 minutes
Fine-tuning (5,343 triplets, 5 epochs)	not feasible	about 10 minutes
Single query against the cached index	under 1 second	—
Benchmark evaluation of one system	about 1 minute	—

A query whose Pankati is already present in the corpus is answered from the cached vector and therefore does not require the embedding model to be loaded at all. This is what allows interactive retrieval to run on a machine that cannot train the model.

6.6 A Methodological Finding on Gurmukhi Tokenisation

Two separate defects arising from the same cause were identified during development, and both are worth recording because they would affect any project applying standard tooling to an Indic script.

Gurmukhi vowel signs such as *ā*, *i* and *u* are encoded as Unicode combining marks. Python's regular expression engine does not classify combining marks as word characters. In consequence, a word boundary assertion never matches after such a mark, and the default token pattern used by scikit-learn's vectoriser, which requires two or more consecutive word characters, retains only runs of consecutive consonants and discards everything else.

The effect is severe. The line *“*ਗਾਵੈ ਕੋ ਵਿਦਿਆ ਵਿਖਮੁ ਵੀਚਾਰੁ'' is reduced by the default tokeniser to the single fragment ਖਮ, as is the entirely unrelated line *“*ਕੋਊ ਬਿਖਮ ਗਾਰ ਤੋਰੈ''. The two lines consequently receive a cosine similarity of exactly 1.000. Both defects were corrected by replacing the default patterns with explicit whitespace and punctuation based tokenisation, after which the mined pairs improved visibly in quality.

6.7 Figures of the Dataset and Implementation

Raw dataset (60,360 rows x 13 columns)

Columns: id, ang, shabad_id, verse_id, pankati, source, is_structural, steek_ss, steek_bdb, steek_ft, steek_ms, steek_pss, raag_id

id	ang	shabad_id	pankati	steek_ss	steek_ms
1	1	1	੧ੑਓ ਸਤਿ ਨਾਮੁ ਕਰਤਾ ਪੁਰਖੁ ਨਿਰਭਉ ਨਿਰਵੈਰੁ ਅਕਾਲ ਮੂਰਤਿ ਅਜੂਨੀ ਸੈਭੰ ਗੁਰ ਪ੍ਰਸਾਦਿ ॥	ਅਕਾਲ ਪੁਰਖ ਇੱਕ ਹੈ, ਜਿਸ ਦਾ ਨਾਮ 'ਹੋਂਦ ਵਾਲਾ' ਹੈ ਜੋ ਸ੍ਰਿਸ਼ਟੀ ਦਾ ...	ਵਾਹਿਗੁਰੂ ਕੇਵਲ ਇਕ ਹੈ। ਸੱਚਾ ਹੈ ਉਸ ਦਾ ਨਾਮ, ਰਚਨਹਾਰ ਉਸ ਦੀ ਵਿਅਕਤ...
2	1	1	॥ ਜਪੁ ॥	ਇਸ ਸਾਰੀ ਬਾਣੀ ਦਾ ਨਾਮ 'ਜਪੁ' ਹੈ ।	ਉਸ ਦਾ ਸਿਮਰਨ ਕਰ।
3	1	1	ਆਦਿ ਸਚੁ ਜੁਗਾਦਿ ਸਚੁ ॥	ਹੇ ਨਾਨਕ ! ਅਕਾਲ ਪੁਰਖ ਮੁੱਢ ਤੋਂ ਹੋਂਦ ਵਾਲਾ ਹੈ, ਜੁਗਾਂ ਦੇ ਮੁੱਢ ਤ...	ਪਰਾਰੰਭ ਵਿੱਚ ਸੱਚਾ, ਯੁਗਾਂ ਦੇ ਸ਼ੁਰੂ ਵਿੱਚ ਸੱਚਾ,
4	1	1	ਹੈ ਭੀ ਸਚੁ ਨਾਨਕ ਹੋਸੀ ਭੀ ਸਚੁ ॥੧॥	ਇਸ ਵੇਲੇ ਭੀ ਮੌਜੂਦ ਹੈ ਤੇ ਅਗਾਂਹ ਨੂੰ ਭੀ ਹੋਂਦ ਵਾਲਾ ਰਹੇਗਾ ।੧।	ਅਤੇ ਸੱਚਾ ਉਹ ਹੁਣ ਭੀ ਹੈ, ਹੇ ਨਾਨਕ! ਨਿਸਚਿਤ ਹੀ, ਉਹ ਸੱਚਾ ਹੋਵੇਗਾ।

Figure 12: Structure of the raw dataset with Pankatis and their commentaries

Cleaned dataset with derived flag columns

Last two rows are structural heading lines flagged by clean_dataset.py and excluded from the corpus.

id	ang	pankati	n_steeks_train	is_header_line	is_duplicate_pankati	is_trainable
1	1	੧ੑਓ ਸਤਿ ਨਾਮੁ ਕਰਤਾ ਪੁਰਖੁ ਨਿਰਭਉ ਨਿਰਵੈਰੁ ਅਕਾਲ ਮੂਰਤਿ ਅਜੂਨੀ ਸੈਭੰ ਗੁਰ ਪ੍ਰਸਾਦਿ ॥	3	0	1	1
3	1	ਆਦਿ ਸਚੁ ਜੁਗਾਦਿ ਸਚੁ ॥	3	0	1	1
4	1	ਹੈ ਭੀ ਸਚੁ ਨਾਨਕ ਹੋਸੀ ਭੀ ਸਚੁ ॥੧॥	3	0	0	1
5	1	ਸੋਚੈ ਸੋਚਿ ਨ ਹੋਵਈ ਜੇ ਸੋਚੀ ਲਖ ਵਾਰ ॥	3	0	0	1
2	1	॥ ਜਪੁ ॥	3	1	0	0
379	8	ਸਲੋਕੁ ॥	1	1	1	0

Figure 13: Cleaned dataset showing the derived flag columns; the final two rows are structural heading lines

Training triplets: anchor, positive, hard negative

Positives agree across two commentators; hard negatives share vocabulary but carry a different meaning.

Anchor Pankati	Positive Pankati	Hard negative	agreement
ਮੰਨੇ ਕੀ ਗਤਿ ਕਹੀ ਨ ਜਾਇ ॥	ਤਾ ਕੀ ਗਤਿ ਮਿਤਿ ਕਥਨੁ ਨ ਜਾਇ ॥੩॥	ਜਾ ਕੀ ਗਤਿ ਮਿਤਿ ਕਹੀ ਨ ਜਾਇ ॥	0.3261
ਮੰਨੈ ਜਮ ਕੈ ਸਾਥਿ ਨ ਜਾਇ ॥	ਆਗੈ ਜਮ ਸਿਉ ਹੋਇ ਨ ਮੇਲ ॥	ਮੰਨੈ ਪਤਿ ਸਿਉ ਪਰਗਟੁ ਜਾਇ ॥	0.3027
ਕੁਦਰਤਿ ਕਵਣ ਕਹਾ ਵੀਚਾਰੁ ॥	ਤਾ ਕੀ ਕੀਮਤਿ ਕਹਣੁ ਨ ਜਾਈ ਕੁਦਰਤਿ ਕਵਨ ਹਮਾ੍ਰੀ ॥੧॥	ਕਵਣ ਮੂਲੁ ਕਵਣ ਮਤਿ ਵੇਲਾ ॥	0.3465
ਕੁਦਰਤਿ ਕਵਣ ਕਹਾ ਵੀਚਾਰੁ ॥	ਤਾ ਕੀ ਕੀਮਤਿ ਕਹਣੁ ਨ ਜਾਈ ਕੁਦਰਤਿ ਕਵਨ ਹਮਾ੍ਰੀ ॥੧॥	ਕਵਣ ਮੂਲੁ ਕਵਣ ਮਤਿ ਵੇਲਾ ॥	0.3531

Figure 14: Mined training triplets of anchor, positive and hard negative

Dataset profile report

```
============================================================
GURBANI DATASET PROFILE
============================================================
Source            : final_gurbani_dataset_all_steeks.xlsx
Total rows        : 60,360
Unique Pankatis   : 55,677

-- exclusions ------------------------------------------------
is_structural = 1       : 720
prefix-rule headings    : 4,880
lexicon-only headings   : 202
rescued by guard        : 5
excluded (total)        : 5,077
TRAINABLE rows          : 55,217

rows sharing text       : 5,235
  ...still in corpus    : 792  (repeated scripture -> Step 2 must group)

-- steek availability (all rows) ----------------------------
steek_ss    :  55,018  [train]
steek_bdb   :  55,034  [excluded from pairs]
steek_ft    :  54,955  [train]
steek_ms    :  60,067  [train]
steek_pss   :  50,015  [excluded from pairs]

ss & bdb both present   : 55,017
  byte-identical        : 54,941 (99.9%)

-- Steeks per TRAINABLE Pankati (ss / ft / ms) ---------------
3 Steeks             :  54,763
2 Steeks             :     253
1 Steeks             :   5,245
0 Steeks             :      99

Estimated Pankati<->Steek positive pairs: 164,898

-- metadata --------------------------------------------------
ang         : 0 missing, 1,429 distinct
shabad_id   : 0 missing, 5,540 distinct
verse_id    : 0 missing, 60,360 distinct
raag_id     : 644 missing, 31 distinct
============================================================
```

Figure 15: Dataset profile report produced by the preprocessing module

Training pair construction report

```
============================================================
TRAINING PAIR CONSTRUCTION
============================================================
Direction  : query = Steek, passage = Pankati
Steeks used: steek_ss, steek_ft, steek_ms
Negatives  : none (in-batch only; hard negatives mined in Step 4)

Trainable rows      : 55,217
Leakage groups      : 5,190
  largest group     : 3,138 rows (5.68% of corpus)

-- split (grouped by shabad_id + repeated Pankati text) ------
train :  44,172 rows  131,806 pairs ( 80.0%)
val   :   5,523 rows   16,457 pairs ( 10.0%)
test  :   5,522 rows   16,515 pairs ( 10.0%)
TOTAL :  55,217 rows  164,778 pairs

-- pairs by Steek source -------------------------------------
  ms  :  54,965
  ss  :  54,954
  ft  :  54,859

dropped, steek < 15 chars : 27
dropped, duplicate pair    : 93

Retrieval corpus (unique Pankatis): 54,844
============================================================
```

Figure 16: Training pair construction report showing the leakage-free partition

Semantic evaluation set report

```
============================================================
SEMANTIC EVALUATION SET  (Pankati -> Pankati)
============================================================
Gold rule   : min(ss_sim, ms_sim) in [0.3, 0.97]
Constraint  : cross-Shabad only; word overlap NOT filtered
Query split : test

Queries offered      : 5,502
Queries with gold    : 827 (15.0%)
Total gold pairs     : 1,758
Gold per query       : mean 2.13  median 1  max 10

-- how much of this is beyond keyword search ------------------
gold pairs with ZERO shared words : 332 (18.9%)
mean word overlap (Jaccard)       : 0.212

-- spread across the Granth ----------------------------------
Ang distance : mean 388  median 324  max 1381
different Raag : 1,505 of 1,758 (85.6%)
============================================================
```

Figure 17: Report of the mined evaluation benchmark

Verse pair mining report

```
============================================================
PANKATI <-> PANKATI TRAINING PAIRS
============================================================
Source split   : train only (no leakage into val/test)
Rule           : min(ss_sim, ms_sim) in [0.3, 0.97]
Constraints    : cross-Shabad, max 3/verse, word overlap <= 0.6

pairs written          : 5,717
distinct anchor verses : 3,737 of 43,821 (8.5%)
dropped, too similar   : 780

commentator agreement  : mean 0.357  median 0.338
word overlap           : mean 0.183
  pairs with ZERO shared words : 1,090 (19.1%)
============================================================
```

Figure 18: Report of the mined Pankati-to-Pankati training pairs

Hard negative mining report

```
============================================================
HARD NEGATIVES  (look alike, mean different things)
============================================================
Rule    : lexical in [0.35, 0.9], meaning <= 0.1, and <= positive's agreement - 0.25
Limit   : 1 negative(s) per pair, cross-Shabad, never a known positive

pairs with a hard negative : 5,343 of 5,717 (93.5%)
pairs left without one     : 374
  ...rejected by margin    : 53

negative lexical similarity : mean 0.534  max 0.895
negative meaning similarity : mean 0.031
============================================================
```

Figure 19: Report of the mined hard negatives

clean_dataset.py — structural line detection

Gurmukhi vowel signs are combining marks, so regex word boundaries never fire after them.

```python
# they repeat verbatim, they poison contrastive training as false negatives.
# ----------------------------------------------------------------------------

# `\b` is unusable here: Gurmukhi vowel signs (ਾਿੀੁੂ) are combining marks, which
# Python's re does not treat as word characters, so a word boundary never fires
# after them. Require an explicit terminator instead.
_END = r"(?=[\s॥।]|$)"

_HEADER_PATTERNS = [
    rf"^ਰਾਗੁ?{_END}",                          # raag heading
    r"^(ਪਉੜੀ|ਸਲੋਕੁ?|ਸਲੋਕ|ਛੰਤੁ?|ਅਸਟਪਦੀ|ਸੋਹਿਲਾ|ਵਾਰ)\s*॥?\s*$",
    rf"^(ਛਕਾ|ਛਕੇ|ਦੁਪਦੇ|ਦੁਪਦਾ|ਚਉਪਦੇ|ਤ੍ਰਿਪਦੇ|ਪੰਚਪਦਾ){_END}",
    r"^ਰਾਗ\s*ਮਾਲਾ",
    # Must be followed by a number ("ਘਰੁ ੨ ॥"). A bare `^ਘਰੁ\s` also swallowed
    # real verses such as "ਘਰੁ ਦਰੁ ਰਾਖਹਿ ਜੇ ਰਸੁ ਚਾਖਹਿ ...".
    r"^ਘਰੁ\s*[O-ੴ0-9]",
    r"^ਮਃ\s*[O-ੴ0-9]",                          # "ਮਃ ੨ ॥"
    r"^ਮਹਲਾ\s*[O-ੴ0-9]",
    r"^[O-ੴ0-9\s॥।]+$",                         # digits / danda only
]
_HEADER_RE = re.compile("|".join(_HEADER_PATTERNS))

# "ਆਸਾ ਮਹਲਾ ੫ ॥", "ਸਾਰਗ ਮਹਲਾ ੫ ॥" — raag + author, only when short.
_AUTHOR_RE = re.compile(r"(ਮਹਲਾ|ਮਃ)\s*[O-ੴ0-9]")
_SHORT_TITLE_MAX = 45


def is_header_line(pankati: str) -> bool:
    if not pankati:
        return True
    # A bare "ੴ ਸਤਿਗੁਰ ਪ੍ਰਸਾਦਿ ॥" is a section header (522 occurrences), but the
    # full Mool Mantar opening the Granth is scripture. Separate them by length.
    if pankati.startswith("ੴ"):
        return len(pankati) <= _SHORT_TITLE_MAX
    if _HEADER_RE.search(pankati):
        return True
    if len(pankati) <= _SHORT_TITLE_MAX and _AUTHOR_RE.search(pankati):
```

Figure 20: Implementation of the structural line detector

create_training_pairs.py — leakage-safe grouping

```python
# fine-tuned score and making the Phase 8 baseline comparison meaningless.
# Union the two relations so the transitive closure is respected.
# ----------------------------------------------------------------------------

class UnionFind:
    def __init__(self):
        self.parent = {}

    def find(self, x):
        self.parent.setdefault(x, x)
        root = x
        while self.parent[root] != root:
            root = self.parent[root]
        while self.parent[x] != root:      # path compression
            self.parent[x], x = root, self.parent[x]
        return root

    def union(self, a, b):
        ra, rb = self.find(a), self.find(b)
        if ra != rb:
            self.parent[ra] = rb


def build_leak_groups(df: pd.DataFrame) -> pd.Series:
    uf = UnionFind()
    for shabad, group in zip(df["shabad_id"], df["pankati_group_id"]):
        uf.union(("S", shabad), ("P", group))

    # Third relation: the same Steek text sometimes annotates two different
    # Pankatis. Without this, an identical query lands in train with one gold
    # Pankati and in val with another -- the model is then scored on a query it
    # was trained to answer differently. Costs ~0.9% on the largest group.
    for col in TRAIN_STEEKS:
        for shabad, steek in zip(df["shabad_id"], df[col]):
            if len(steek) >= MIN_STEEK_CHARS:
                uf.union(("S", shabad), ("T", steek))

    # A root may be a ("T", <long steek>) key, so map roots to compact integer
    # ids -- deterministic, and keeps the JSONL records small.
    roots = df["shabad_id"].map(lambda s: uf.find(("S", s)))
    ids = {root: i for i, root in enumerate(sorted(set(roots), key=str))}
```

Figure 21: Implementation of the leakage-free partitioning procedure

mine_verse_pairs.py — mining Pankati-to-Pankati positives

```
        for r, qi in enumerate(block):
            scores = sim[r]
            scores[shabad == shabad[qi]] = -1.0       # cross-Shabad only
            hits = np.flatnonzero((scores >= THRESHOLD) & (scores <= CEILING))
            if hits.size == 0:
                continue
            hits = hits[np.argsort(-scores[hits])]
            kept = 0
            for j in hits:
                if kept >= MAX_PER_VERSE:
                    break
                key = (min(qi, j), max(qi, j))
                if key in seen_pairs:
                    continue
                overlap = jaccard(pankati[qi], pankati[j])
                # A positive pair that already shares most of its words teaches
                # the shortcut we are trying to unlearn.
                if overlap > MAX_OVERLAP:
                    dropped_overlap += 1
                    continue
                seen_pairs.add(key)
                kept += 1
                records.append({
                    "query": pankati[qi],
                    "pos": [pankati[j]],
                    "neg": [],
                    "pair_type": "verse_verse",
                    "agreement": round(float(scores[j]), 4),
                    "word_overlap": round(overlap, 4),
                    "shabad_id": int(shabad[qi]),
                })
        print(f"  {min(start + CHUNK, len(pool)):,}/{len(pool):,}", end="\r")

    print()
    with OUT_JSONL.open("w", encoding="utf-8") as f:
```

Figure 22: Implementation of the Pankati-to-Pankati pair mining

train_bge.py — LoRA fine-tuning of BGE-M3

```
from datasets import Dataset
from sentence_transformers import (
    SentenceTransformer, SentenceTransformerTrainer,
    SentenceTransformerTrainingArguments,
)
try:                                    # moved in sentence-transformers 6.x
    from sentence_transformers.sentence_transformer import losses
except ImportError:
    from sentence_transformers import losses

import transformers
global _TRANSFORMERS_V5
_TRANSFORMERS_V5 = int(transformers.__version__.split(".")[0]) >= 5

random.seed(SEED)
torch.manual_seed(SEED)

pair_files = [PROCESSED / n.strip() for n in args.pairs.split(",") if n.strip()]
print("training pairs:")
train = load_pairs(pair_files, args.max_pairs)
val = load_pairs(PROCESSED / "val_pairs.jsonl", 2000)
print(f"train pairs: {len(train['anchor']):,}   val pairs: {len(val['anchor']):,}")

print(f"loading {args.base_model} ...")
model = SentenceTransformer(args.base_model)
model.max_seq_length = MAX_SEQ

transformer = model[0].auto_model

if args.lora:
    try:
        from peft import LoraConfig
    except ImportError:
        sys.exit("--lora needs peft:  pip install peft")
    # Attention projections only. "dense" would match 73 modules in
    # XLM-RoBERTa (attention output, both FFN layers, and the unused
```

Figure 23: Implementation of the LoRA fine-tuning procedure

7. Output Screens

This chapter presents the output of the working retrieval system. The interface is at present a command-line application; the graphical user interface described in the synopsis is scheduled for the remaining phase of the project.

7.1 Retrieval Output

Retrieval output of the fine-tuned model

```
(.venv) E:\Major_Project_Gurveer> python src\retrieval\semantic_search.py --tag verse
corpus: 54,844 Pankatis

Pankati (or 'q' to quit) > ਆਨ ਨ ਬੀਆ ਤੇਰੀ ਸਮਸਰਿ ॥

QUERY: ਆਨ ਨ ਬੀਆ ਤੇਰੀ ਸਮਸਰਿ ॥

========================================================================
model: verse
========================================================================

 1. [0.746]  ਤੁਮ ਸਮਸਰਿ ਅਵਰੁ ਕੋ ਨਾਹੀ ॥੬॥
    Ang 416 (+45)   Raag: Raag Aasaa   Shabad 1607
    ਅਰਥ (SS): ਤੇਰੇ ਵਰਗਾ (ਰਾਖਾ) ਹੋਰ ਕੋਈ ਨਹੀਂ ਹੈ ।੬।

 2. [0.735]  ਤੁਮ ਸਮਸਰਿ ਨਾਹੀ ਦਇਆਲੁ ਮੋਹਿ ਸਮਸਰਿ ਪਾਪੀ ॥੪॥੩॥
    Ang 856 (+485)   Raag: Raag Bilaaval   Shabad 3206
    ਅਰਥ (SS): ਤੇਰੇ ਜੇਡਾ ਕੋਈ ਦਇਆ ਕਰਨ ਵਾਲਾ ਨਹੀਂ, ਤੇ ਮੇਰੇ ਵਰਗਾ ਕੋਈ ਪਾਪੀ ਨਹੀਂ ।੪।੩।

 3. [0.706]  ਹਰਿ ਕੇ ਨਾਮ ਸਮਸਰਿ ਕਛੁ ਨਾਹਿ ॥
    Ang 265 (+106)   Raag: Raag Gauree   Shabad 891
    ਅਰਥ (SS): (ਅਜੇਹਾ) ਕੋਈ (ਉੱਦਮ) ਪ੍ਰਭੂ ਦੇ ਨਾਮ ਦੇ ਬਰਾਬਰ ਨਹੀਂ ਹੈ;

 4. [0.664]  ਆਨ ਨ ਸਮਸਰਿ ਕੋਊ ਲਾਗੈ ਢੂਢਿ ਰਹੇ ਹਮ ਮੁਚਾ ॥੧॥ ਰਹਾਉ ॥
    Ang 534 (+163)   Raag: Raag Dayv Gandhaaree   Shabad 2053
    ਅਰਥ (SS): ਮੈਂ ਬਹੁਤ ਭਾਲ ਕਰ ਕੇ ਥੱਕ ਗਿਆ ਹਾਂ । ਕੋਈ ਹੋਰ ਉਸ ਦੀ ਬਰਾਬਰੀ ਨਹੀਂ ਕਰ ਸਕਦਾ ।੧।ਰਹਾਉ।
```

Figure 24: Retrieval output for a query Pankati, showing similarity score, Ang, Raag, Shabad and Steek for each result

7.2 Comparison of the Pretrained and Fine-tuned Models

Same query answered by the fine-tuned and the pretrained model

```
QUERY: ਭਾਉ ਕਰਮ ਕਰਿ ਜੰਮਸੀ ਸੇ ਘਰ ਭਾਗਠ ਦੇਖੁ ॥੧॥

========================================================================
model: verse   (fine-tuned on Pankati-to-Pankati pairs)
========================================================================
 1. [0.553]  ਕਿਰਸਾਣੀ ਜੰਮੀ ਭਾਉ ਕਰਿ ਹਰਿ ਬੋਹਲ ਬਖਸ ਜਮਾਇਆ ॥
    Ang 304   Raag: Raag Gauree
    ਅਰਥ (SS): ਫ਼ਸਲ ਪ੍ਰੇਮ ਨਾਲ ਉੱਗਦੀ ਹੈ ਤੇ ਪ੍ਰਭੂ ਦੀ ਮਿਹਰ-ਰੂਪੀ ਬੋਹਲ ਦਾ ਢੇਰ ਲੱਗ ਜਾਂਦਾ ਹੈ ।

 2. [0.551]  ਅੰਨੁ ਜੰਮਿਆ ਖੇਤੀ ਭਾਉ ਕਰਿ ਹਰਿ ਨਾਮੁ ਸਮ੍ਹਾਰੇ ॥
    Ang 1318   Raag: Raag Kaanraa
    ਅਰਥ (SS): ਪ੍ਰੇਮ (-ਰੂਪ) ਵਾਹੀ ਕਰਨ ਨਾਲ ਪਰਮਾਤਮਾ ਦਾ ਨਾਮ-ਫ਼ਸਲ ਉੱਗ ਪੈਂਦਾ ਹੈ ।

 5. [0.520]  ਭਾਗਠੜੇ ਹਰਿ ਸੰਤ ਤੁਮ੍ਹਾਰੇ ਜਿਨ੍ ਘਰਿ ਧਨੁ ਹਰਿ ਨਾਮਾ ॥
    Ang 749   Raag: Raag Soohee

========================================================================
model: baseline   (pretrained, no Gurbani training)
========================================================================
 1. [0.748]  ਕਾਰਜੁ ਸਗਲ ਅਰੰਭਿਓ ਘਰ ਕਾ ਠਾਕੁਰ ਕਾ ਭਾਰੇਸਾ ॥੧॥
    Ang 1266   Raag: Raag Malaar

 2. [0.747]  ਅਪਨੀ ਪਤਿ ਸੇਤੀ ਘਰਿ ਜਾਵਹੁ ॥੧॥ ਰਹਾਉ ॥
    Ang 1254   Raag: Raag Malaar

 5. [0.735]  ਕੂੜੁ ਕਮਾਵਹਿ ਆਦਮੀ ਬਾਂਧਹਿ ਘਰ ਬਾਰਾ ॥੧॥
    Ang 418   Raag: Raag Aasaa
```

Figure 25: Side-by-side output of the pretrained and fine-tuned models for the same query

7.3 Training Output

Fine-tuning run showing falling training and validation loss

```
(.venv) E:\Major_Project_Gurveer> python src\training\train_bge.py --pairs verse_pairs_hn.jsonl --lora --batch-size 64 --epochs 5
method: LoRA   lr=0.0001   batch=64
GPU: NVIDIA GeForce RTX 3060  12.9 GB
training pairs:
  verse_pairs_hn.jsonl: 5,343 pairs
  -> using explicit hard negatives (5,343)
train pairs: 5,343   val pairs: 2,000
loading BAAI/bge-m3 ...
LoRA: 2,359,296 trainable of 570,114,048 (0.41%)
{'loss': '1.432', 'grad_norm': '3.102', 'learning_rate': '7.778e-05', 'epoch': '0.08'}
{'loss': '0.7968', 'grad_norm': '3.736', 'learning_rate': '9.359e-05', 'epoch': '0.16'}
{'loss': '0.6460', 'grad_norm': '4.121', 'learning_rate': '8.470e-05', 'epoch': '0.24'}
{'loss': '0.5463', 'grad_norm': '3.580', 'learning_rate': '7.580e-05', 'epoch': '0.32'}
{'loss': '0.5061', 'grad_norm': '5.660', 'learning_rate': '5.801e-05', 'epoch': '0.48'}
{'loss': '0.4615', 'grad_norm': '8.006', 'learning_rate': '4.021e-05', 'epoch': '0.64'}
{'loss': '0.4530', 'grad_norm': '3.749', 'learning_rate': '3.132e-05', 'epoch': '0.72'}
{'eval_loss': '0.3175', 'eval_runtime': '27.99', 'epoch': '0.80'}
{'loss': '0.4476', 'grad_norm': '4.524', 'learning_rate': '1.352e-05', 'epoch': '0.88'}
{'eval_loss': '0.3010', 'eval_runtime': '11.42', 'epoch': '1.00'}
{'train_runtime': '326.7', 'train_samples_per_second': '61.23', 'train_loss': '0.6024'}
probe drift: 0.041872  (0 = model output unchanged)
reload check: cosine 1.000000  (1.0 = saved correctly)

Saved to models\bge-m3-gurbani
```

Figure 26: Fine-tuning run showing the falling training and validation loss

7.4 Evaluation Output

Evaluation of all retrieval systems on the benchmark

```
(.venv) E:\Major_Project_Gurveer> python src\evaluation\evaluate.py --bm25 --bm25-char --dense baseline --dense finetuned --dense verse
eval queries: 919
corpus: 54,844 unique Pankatis
building BM25 (word) ...
building BM25 (char 3-5gram) ...
dense 'baseline': (54844, 1024)
dense 'finetuned': (54844, 1024)
dense 'verse': (54844, 1024)

============================================================================
PANKATI -> PANKATI RETRIEVAL  (same-Shabad hits excluded)
============================================================================
queries: 919    corpus: 54,844

model                      H@1     H@5    H@10    H@50     MRR  nDCG@10
----------------------------------------------------------------------
BM25 (word)               8.92   20.02   23.18   37.76   13.92    11.95
BM25 (char 3-5gram)       9.47   20.46   27.09   44.07   15.01    13.51
dense: baseline           7.83   15.13   19.37   30.25   11.51     9.98
dense: finetuned          8.16   14.80   18.50   30.69   11.64     9.80
dense: verse             11.43   21.65   28.29   45.27   16.77    14.38

H@k = % of queries with at least one gold verse in the top k

-- the subset keyword search cannot reach ---------------------------------
queries whose gold includes a ZERO-word-overlap verse: 420
  BM25 (word)            hit@10 on those:   0.00%
  BM25 (char 3-5gram)    hit@10 on those:   1.19%
  dense: baseline        hit@10 on those:   1.43%
  dense: finetuned       hit@10 on those:   1.67%
  dense: verse           hit@10 on those:   3.57%
============================================================================
```

Figure 27: Evaluation output comparing all retrieval systems on the benchmark

7.5 Corpus Encoding Output

Corpus encoding with shard checkpointing

```
(.venv) E:\Major_Project_Gurveer> python src\retrieval\build_index.py --tag baseline
corpus: 54,844 unique Pankatis
0/28 shards already done
loading BAAI/bge-m3 ... (first run downloads ~2.3 GB)
  shard  1/28    3.6%  elapsed 5.2m   eta 140.8m
  shard  5/28   17.9%  elapsed 23.7m  eta 109.0m
  shard 10/28   35.7%  elapsed 44.2m  eta 79.6m
  shard 15/28   53.6%  elapsed 66.5m  eta 57.6m
  shard 20/28   71.4%  elapsed 84.8m  eta 33.9m
  shard 25/28   89.3%  elapsed 102.8m eta 12.3m
  shard 28/28  100.0%  elapsed 112.7m eta 0.0m

Wrote data\embeddings\baseline\embeddings.npy  shape=(54844, 1024)
Shards kept for resume; delete part_*.npy to reclaim space.
```

Figure 28: Corpus encoding with shard checkpointing and progress reporting

References

[1] F. Feng, Y. Yang, D. Cer, N. Arivazhagan, and W. Wang, "Language-agnostic bert sentence embedding," in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics*, 2022, pp. 878–891.

[2] L. Wang, N. Yang, X. Huang, L. Yang, R. Majumder, and F. Wei, "Multilingual e5 text embeddings: A technical report," *arXiv preprint arXiv:2402.05672*, 2024.

[3] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "Lora: Low-rank adaptation of large language models," in *International Conference on Learning Representations (ICLR)*, 2022.

[4] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, and Z. Liu, "Bge m3-embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation," in *Findings of the Association for Computational Linguistics: ACL 2024*, 2024, pp. 2318–2335.

[5] V. Karpukhin, B. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-T. Yih, "Dense passage retrieval for open-domain question answering," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, 2020, pp. 6769–6781.

[6] M. Douze, A. Guzhva, C. Deng, J. Johnson, G. Szilvasy, P.-E. Mazaré, M. Lomeli, L. Hosseini, and H. Jégou, "The faiss library," *arXiv preprint arXiv:2401.08281*, 2024.

[7] D. Kakwani, A. Kunchukuttan, S. Golla, G. N. C., A. Bhattacharyya, M. M. Khapra, and P. Kumar, "Indicnlpsuite: Monolingual corpora, evaluation benchmarks and pre-trained multilingual language models for indian languages," in *Findings of the Association for Computational Linguistics: EMNLP 2020*, 2020, pp. 4948–4961.

[8] G. Ramesh, S. Doddapaneni, A. Bheemaraj, M. Jobanputra, R. AK, A. Sharma, S. Sahoo, H. Diddee, M. J., D. Kakwani, N. Kumar, A. Pradeep, S. Nagaraj, K. Deepak, V. Raghavan, A. Kunchukuttan, P. Kumar, and M. S. Khapra, "Samanantar: The largest publicly available parallel corpora collection for 11 indic languages," *Transactions of the Association for Computational Linguistics*, vol. 10, pp. 145–162, 2022.

[9] S. Deode, J. Gadre, A. Kajale, A. Joshi, and R. Joshi, "L3cube-indicsbert: A simple approach for learning cross-lingual sentence representations using multilingual bert," *arXiv preprint arXiv:2304.11434*, 2023.

[10] N. Reimers and I. Gurevych, "Sentence-bert: Sentence embeddings using siamese bert-networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, 2019, pp. 3982–3992.

[11] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: Bm25 and beyond," in *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, 2009, pp. 333–389.